PRACTICAL WORK (TP) — SERIES No. 2

Getting Started with Python

Student version

Module	Level	Duration	Tools	Academic year
Informatique 1 (UE Méthodologie)	L1 Common Core Sciences de la Matière	3 sessions × 1 h 30	Python 3 + IDLE	2026 – 2027

1. Presentation of the series

This series follows Chapter 2 of the Informatique 1 programme (**Débuter avec Python**). It takes the student from a first one-line program to small scientific calculators (area of a triangle, molar concentration of a solution). Each exercise gives a statement, objectives, test data with the expected results, and an optional extension. Write your algorithm and your program yourself, then compare your results with the expected ones.

Learning outcomes

- write, save and run a Python script in IDLE;
- display results with print() and read data from the keyboard with input();
- create variables and constants, choose valid names, and identify data types with type();
- convert between types (int(), float(), str()) and use arithmetic operators + - * / // % **;
- translate a mathematical or chemical formula into a Python expression (units included);
- read a Python error message and correct the program.

Session plan

Session	Theme	Exercises	Key skills	Time
1	First programs and input / output	1 – 5	print, comments, input, int/float, simple arithmetic	90 min
2	Variables, constants and data types	6 – 8	assignment, naming rules, type(), conversions, constants, **	90 min
3	Scientific applications	9 – 11	math.sqrt, formulas, units, round, formatted output (f-strings)	90 min

Working method (for every exercise)

1. Read the statement twice; identify the **data** (inputs), the **results** (outputs) and the **formula**.
2. Write the **algorithm** on paper (START – READ – COMPUTE – DISPLAY – END).
3. In IDLE: **File → New File**, type the program, save it as tp2_ex1.py, tp2_ex2.py, …
4. Run with **Run → Run Module (F5)** and read the output in the Shell.
5. **Test** with at least two data sets, one of them a limit case (zero, negative number, decimal).

> **Good practice.** Use meaningful variable names, add a comment line at the top of each file (exercise number, author, date), indent with 4 spaces, and always mention the **unit** in the prompt (e.g. “Volume V (L)”).

2. Quick reference (Python memento)

Basic syntax

Element	Example	Remark
Comment	# this line is ignored	Everything after # is ignored by Python.
Display	print("Hello", x)	Several items are separated by a space; sep= and end= change this.
Keyboard input	name = input("Name: ")	**Always returns a string (`str`)**, even if digits are typed.
Assignment	age = 19	= stores a value in a variable; it is not a mathematical equality.
Constant (convention)	PI = 3.14159	Upper-case name; Python does not forbid changing it.
Import a module	import math	Then math.sqrt(x) and math.pi become available.

Fundamental data types

Type	Meaning	Examples	type() shows
int	integer	25, -7, 0	<class 'int'>
float	real (decimal) number — use a **dot**	3.14, -0.5, 2.0	<class 'float'>
str	text (string)	"Python", '25'	<class 'str'>
bool	logical value	True, False	<class 'bool'>
complex	complex number	2 + 3j	<class 'complex'>

Arithmetic operators and useful functions

Operator / function	Meaning	Example	Result
+ - *	add, subtract, multiply	7 * 3	21
/	real division (always a float)	7 / 2	3.5
//	integer (floor) division	7 // 2	3
%	remainder (modulo)	7 % 2	1
**	power	2 ** 3	8
int(x) float(x) str(x)	type conversion	int("7") + 1	8
round(x, n)	round to n decimals	round(3.14159, 2)	3.14
math.sqrt(x)	square root	math.sqrt(9)	3.0

Priority of operators: parentheses ( ) → power ** → *, /, //, % → +, -. Example: 2 + 3 * 4 ** 2 = 50.
Valid variable names: letters, digits and _; must not start with a digit; no spaces; not a Python keyword (class, if, for…); upper/lower case are different (Age ≠ age). Style: snake_case for variables, UPPER_CASE for constants.
Formatted output (f-string): print(f"Area = {area:.2f} cm²") displays area with 2 decimals.

3. Session 1 — First programs and input / output

Exercises 1 to 5 · 90 minutes · Skills: print, comments, input, conversions, simple arithmetic.

Exercise 1 — Hello, World!

STATEMENT

> Write a Python program that displays on the screen the message:
> Hello, World!

OBJECTIVES

- open IDLE and run a first program;
- use the print() function and understand what a string literal is.

YOUR ALGORITHM

YOUR PROGRAM

Save it as tp2_ex1.py, run it with F5 and test it.

TEST DATA AND EXPECTED RESULTS

- Display: Hello, World!

GOING FURTHER (OPTIONAL)

> Display the message on **two lines** with a single print call (hint: the special character \n means “new line”).

Exercise 2 — My first program

STATEMENT

> Write a Python program that displays on the screen the message:
> This is my first program in Python
> Start the file with two comment lines giving the exercise number, your name and the date. Save it as tp2_ex2.py and run it.

OBJECTIVES

- write comments with #;
- save a script in a .py file and run it with **F5**;
- use several arguments in print() with sep and end.

YOUR ALGORITHM

YOUR PROGRAM

Save it as tp2_ex2.py, run it with F5 and test it.

TEST DATA AND EXPECTED RESULTS

- Display: This is my first program in Python

GOING FURTHER (OPTIONAL)

Display the same sentence using three separate arguments, then using two print calls on the **same line**, and finally display the date 2026-10-07 with sep="-".

Exercise 3 — Sum of two integers

STATEMENT

Write a Python program that reads two **integers** a and b entered by the user, computes their sum and displays it.

Test the program with a = 12, b = 30, then with negative numbers.

OBJECTIVES

- read data with input() and convert it with int();
- store a result in a variable and display it.

YOUR ALGORITHM

YOUR PROGRAM

Save it as tp2_ex3.py, run it with F5 and test it.

TEST DATA AND EXPECTED RESULTS

- a = 12, b = 30 → 42
- a = -7, b = 4 → -3

GOING FURTHER (OPTIONAL)

Extend the program: for two integers a and b (with $b \neq 0$) display the sum, difference, product, integer quotient (//) and remainder (%). Test with a = 17, b = 5.

Exercise 4 — Sum, product and division of two real numbers

STATEMENT

> Write a Python program that reads two **real numbers** x1 and x2 (with $x1 \neq 0$ and $x2 \neq 0$) and displays:
> • their sum som • their product prod • their quotient div = x1 / x2.
> Test with x1 = 7.5 and x2 = 2.5.

OBJECTIVES

- read real numbers with float();
- use +, * and / and observe that / always returns a float;
- understand why division by zero must be avoided.

YOUR ALGORITHM

YOUR PROGRAM

Save it as tp2_ex4.py, run it with F5 and test it.

TEST DATA AND EXPECTED RESULTS

- x1 = 7.5, x2 = 2.5 → sum 10.0, product 18.75, division 3.0

GOING FURTHER (OPTIONAL)

> **(a)** Display the quotient and the arithmetic mean (x1 + x2) / 2 with **2 decimals** for x1 = 1, x2 = 3.
> **(b)** Make the program robust: if x2 equals 0, display a message instead of crashing (preview of Chapter 3 — conditional structures).

Exercise 5 — Name and age of the user

STATEMENT

> Write a Python program that asks the user for their **name** and **age**, then displays them in a sentence.

OBJECTIVES

- combine text input and numeric input in one program;
- display text and variables together.

YOUR ALGORITHM

YOUR PROGRAM

Save it as tp2_ex5.py, run it with F5 and test it.

TEST DATA AND EXPECTED RESULTS

- Name Amina, age 19 → Hello, Amina then You are 19 years old.

GOING FURTHER (OPTIONAL)

Use an **f-string** to display one sentence, and compute the approximate year of birth (take the current year as the constant CURRENT_YEAR = 2026).

Session 1 wrap-up

- I can write, save and run a .py script.
- I know that input() returns a string and I convert it with int() or float().
- I can display text and values with print(), sep, end and f-strings.
- I can recognise NameError, SyntaxError, ValueError and TypeError.

4. Session 2 — Variables, constants and data types

Exercises 6 to 8 · 90 minutes · Skills: assignment, naming rules, type(), conversions, constants, powers.

Exercise 6 — Creating and displaying variables

STATEMENT

> Create three variables: name (a first name), age (a whole number) and length (the height of a person in metres, a real number). Then display them.

OBJECTIVES

- assign values to variables of different kinds;
- respect the naming rules;
- understand that a variable can be re-assigned.

YOUR ALGORITHM

YOUR PROGRAM

Save it as tp2_ex6.py, run it with F5 and test it.

TEST DATA AND EXPECTED RESULTS

- Name Amina, age 19, length 1.65 displayed on three lines

GOING FURTHER (OPTIONAL)

> **Swap** the contents of two variables a = 3 and b = 7 (a) with a temporary variable, (b) with Python's simultaneous assignment.

Exercise 7 — Data types

STATEMENT

> Define variables of **five different data types** (integer, real, string, boolean, complex), then display their values and their types.

OBJECTIVES

- recognise the fundamental types int, float, str, bool, complex;
- use type() to check the type of a value.

YOUR ALGORITHM

YOUR PROGRAM

Save it as tp2_ex7.py, run it with F5 and test it.

TEST DATA AND EXPECTED RESULTS

- Each line shows the value and its type, e.g. 25 <class 'int'>; the five types int, float, str, bool, complex must appear

GOING FURTHER (OPTIONAL)

Predict then check the results of these **type conversions**, and of the two divisions 5 / 2 and 5 // 2.

Exercise 8 — Area of a circle (constants)

STATEMENT

Define the constant PI = 3.14159 and the radius R = 1.5 (cm) of a circle. Compute and display its area A = PI × R².

OBJECTIVES

- use upper-case names for constants (convention);
- write a formula with the power operator **;
- attach a unit to a result and round it.

YOUR ALGORITHM

YOUR PROGRAM

Save it as tp2_ex8.py, run it with F5 and test it.

TEST DATA AND EXPECTED RESULTS

- R = 1.5 cm → area 7.0685775 cm² (rounded to 2 decimals: 7.07)

GOING FURTHER (OPTIONAL)

Add the circumference 2·PI·R and the volume of a sphere of the same radius 4/3·PI·R³, and compare the area with the one obtained with math.pi.

Session 2 wrap-up

- I can choose valid, meaningful variable names and I know the convention for constants.
- I can name the five fundamental types and check them with type().
- I can convert between int, float, str and bool and I know that int(3.99) gives 3.
- I can write a formula with ** and round a result with round().

5. Session 3 — Scientific applications

Exercises 9 to 11 · 90 minutes · Skills: math.sqrt, formulas with units, rounding, formatted output.

Exercise 9 — Area of a triangle (Heron's formula)

STATEMENT

> Write a program that reads the three sides a, b, c of a triangle (in the same unit) and computes its area with **Heron's formula**:
>
> $p = (a + b + c) / 2$ and $\text{area} = \sqrt{p\,(p-a)(p-b)(p-c)}$
>
> Test with the sides (3, 4, 5), then (7, 8, 9).

OBJECTIVES

- import and use the math module (math.sqrt);
- translate a two-step formula into several Python statements;
- check by hand that the result is correct.

YOUR ALGORITHM

YOUR PROGRAM

Save it as tp2_ex9.py, run it with F5 and test it.

TEST DATA AND EXPECTED RESULTS

- Sides 3, 4, 5 → p = 6.0, area 6.0
- Sides 7, 8, 9 → p = 12.0, area ≈ 26.83
- Sides 1, 2, 5 → not a triangle

GOING FURTHER (OPTIONAL)

> For the triangle (3, 4, 5), compute the radius of the inscribed circle r = area / p and the height relative to side c: h = 2·area / c.

Exercise 10 — Number of moles and molar concentration

STATEMENT

A mass m (g) of a solute of molar mass M (g/mol) is dissolved to make a volume V (L) of solution. Write a program that reads m, M and V, then computes and displays:

• the amount of substance $n = m / M$ (mol)

• the molar concentration $C = n / V$ (mol/L)

Test with 5.85 g of NaCl (M = 58.5 g/mol) in 0.5 L of solution.

OBJECTIVES

- implement chemical formulas with consistent units;
- chain two calculations (the result n is reused to compute C);
- understand rounding of floating-point results.

YOUR ALGORITHM

YOUR PROGRAM

Save it as tp2_ex10.py, run it with F5 and test it.

TEST DATA AND EXPECTED RESULTS

- m = 5.85 g, M = 58.5 g/mol, V = 0.5 L → n = 0.1 mol, C = 0.2 mol/L

GOING FURTHER (OPTIONAL)

(a) What mass of NaOH (M = 40 g/mol) must be weighed to prepare 250 mL (0.25 L) of a solution at 0.1 mol/L? Use $m = C \cdot V \cdot M$.

(b) What volume of a stock solution at 1 mol/L must be taken to prepare 100 mL of a solution at 0.1 mol/L? Use the dilution relation $C1 \cdot V1 = C2 \cdot V2$.

Exercise 11 — Synthesis — laboratory report (new)

STATEMENT

Write a program that reads the **name** of a substance, its **mass m** (g), its **molar mass M** (g/mol) and the **volume V in millilitres**. The program converts the volume into litres, computes n and C, and displays a neat report.

Test with NaCl, 5.85 g, 58.5 g/mol, 500 mL.

OBJECTIVES

- combine input, conversions, calculations and formatted output in one program;
- convert units (mL → L);

- format numbers with f-strings.

YOUR ALGORITHM

YOUR PROGRAM

Save it as tp2_ex11.py, run it with F5 and test it.

TEST DATA AND EXPECTED RESULTS

- NaCl, 5.85 g, 58.5 g/mol, 500 mL → V = 0.500 L, n = 0.1000 mol, C = 0.2000 mol/L

GOING FURTHER (OPTIONAL)

> Add a line to the report giving the concentration in mmol/L (C × 1000).

Session 3 wrap-up

- I can import the math module and use math.sqrt.
- I can turn a scientific formula into Python, checking the **units** and the **priority of operators**.
- I can round and format results (round, f-strings).
- I can test my program with several da-ta sets and verify one result by hand.

6. Check your understanding

Predict the output of each instruction **without running it**, then check in IDLE.

#	Instruction(s)	Your prediction
1	print(5 // 2, 7 % 3, 2 ** 3, 7 / 2)	
2	print("3" + "4", "3" * 3)	
3	print(int("7") + 1)	
4	print(type(True), type(3.0), type("3"))	
5	x = 10 x = x + 5 print(x)	
6	print(round(3.14159, 2), abs(-4), max(2, 9, 4))	
7	print(10 / 2)	
8	print("a", "b", sep="-")	

Annex A — Common error messages

The exact wording may differ slightly between Python versions (3.10 to 3.13), but the error type is always the same.

Error type and message	Example	Probable cause	How to fix
NameError: name 'Print' is not defined	Print("hi")	misspelled name, wrong case, variable not yet defined, missing quotes or missing import	check spelling; define the variable before using it; add quotes or import
SyntaxError: unterminated string literal	print("hi)	closing quote or parenthesis missing; misuse of symbols	close the quote; read the line indicated by IDLE
IndentationError: unexpected indent	a line starting with spaces	spaces at the start of a line that should not be indented	remove the spaces
ValueError: invalid literal for int() with base 10: 'abc'	int("abc")	text that cannot be converted to a number (or 3.5 for int)	type a valid number; use float for decimals
TypeError: can only concatenate str (not "int") to str	"a" + 1	mixing text and number	convert with int() / str() or use commas in print
ZeroDivisionError: division by zero	5 / 0	denominator equal to zero	check the data before dividing

Annex B — Assessment grid (TP report, /20)

Criterion	Description	Points
Correct results	The program gives the expected outputs on the test data (including a limit case)	8
Use of the right concepts	Appropriate conversions (int, float), correct operators and formulas, units respected	4
Code quality	Meaningful names, comments, indentation, readable output	4
Tests and analysis	At least two data sets tested; one result verified by hand; errors explained	2
Autonomy and punctuality	Work done in the session, files saved and named correctly	2
Total		**20**